

VARUNA - Round 2 Work Distribution

SIH26067 · 4-day core build (Sept 5 – Sept 8) · UI/Frontend deferred to Day 4

How this is organized

- 6 people, 6 tracks, running in parallel from Day 1.
 - Everyone builds against an agreed API contract from Day 1, so nobody blocks anybody.
 - UI/Visualization is intentionally **not** touched until Day 4 — the backend + intelligence loop must work end-to-end (testable via curl/Postman) before anyone opens CesiumJS.
 - Every person must be able to explain, modify, and defend every part of the system, not just their own track — judges test this directly.
-

Person 1 — Graph Fusion Engine (Intelligence Core #1)

Owens: The engine that corrects the model grid using a graph of real sensor readings. This is the primary technical differentiator judges will interrogate hardest.

Tech: PyTorch Geometric (fallback: manual graph-weighted correction with `networkx` if training proves unstable)

Day	Task
1	Design the graph structure for the sensor network (nodes = in-situ sensors, edges weighted by real oceanographic connectivity, not straight-line distance)
2	Build graph-weighted correction against a mock model grid; stub the <code>/fused</code> endpoint
3	Wire the engine to Person 4's real <code>/model</code> output and Person 6's real <code>/observations</code> output → produce a real <code>/fused</code> response
4	Bug fixes, stability pass, threshold sanity checks. Support the frontend team's questions on trust-overlay data shape. Help prep pitch script / backup video.

Never cut: at least one visible fusion signal — this is the non-negotiable floor of the whole demo. **Fallback if behind:** drop full PyG GNN training, ship the simpler graph-weighted correction — it's still legitimately "graph-based fusion" under questioning.

Person 2 — Drift Memory Engine (Intelligence Core #2)

Owens: The rolling per-region "surprise" score that flags model-vs-reality divergence live. Second core differentiator.

Day	Task
1	Define the surprise-score formula and the divergence threshold logic that will trigger alerts
2	Compute divergence score against mock model + mock observation data; stub the <code>/alerts</code> endpoint
3	Wire Drift Memory to Person 1's real <code>/fused</code> output → produce real <code>/alerts</code> ; tune the threshold with real numbers
4	Bug fixes, threshold tuning based on integration test results. Support frontend team on alert-feed data shape (e.g. "Drift detected: Bay of Bengal, SST diverging 1.8°C from forecast"). Help prep pitch script / backup video.

Never cut: this is the second non-negotiable piece of the core loop.

Person 3 — Streaming Backbone + Nowcast Engine (stretch)

Owns: The real-time ingestion state-update logic, and the optional short-range Nowcast prediction if time allows.

Tech: Plain LSTM / small custom linear-time recurrent function (~20 lines PyTorch/NumPy, CPU-only, no compiled extensions — this is deliberate, see note below). Nowcast Engine is ConvLSTM-inspired (Shi et al. 2015, precipitation-nowcasting lineage), not the JEPA-based version from the original draft.

Day	Task
1	Scaffold the ~20-line LSTM state-update function; review the ConvLSTM approach for the Nowcast stretch goal
2	Get the LSTM streaming ingestion working end-to-end against a mock data stream
3	Attempt the ConvLSTM Nowcast Engine if ahead of schedule; otherwise pivot to supporting integration work across other tracks
4	Bug fixes, stability pass. If Nowcast isn't ready, it becomes a documented "Future Scope" slide bullet — that's an acceptable, planned outcome, not a failure. Help prep pitch script / backup video.

Important note for this track: the architecture diagram shows Mamba/SSM as the target-state design (for finals), but **do not install the `mamba-ssm` package** — it needs CUDA kernels that don't build reliably on Windows. The actual Sept 8 build uses the plain LSTM described above. Same architectural story on the slide, zero install risk.

Person 4 — Ocean Model Data Pipeline

Owens: Ingesting and regridding the simulated/model ocean data — the "Version A" side of the digital twin.

Sources: INCOIS LAS (primary) + Copernicus Marine GLORYS reanalysis (fallback if LAS access/format issues come up) **Variables:** Sea Surface Temperature, Currents (u/v), Wave Height, Salinity, Chlorophyll **Tech:** xarray + netCDF4, regrid to common resolution

Day	Task
1	Stand up INCOIS LAS ingestion, confirm Copernicus GLORYS fallback path works; start regridding pipeline
2	Get real NetCDF-derived data (all 5 variables) serving through Person 5's <code>/model</code> endpoint
3	Support Person 1 in wiring Graph Fusion to real <code>/model</code> data; fix any format/unit mismatches that surface during integration
4	Bug fixes, data-quality pass. Likely one of the 3 people pulled onto frontend for Day 4 (see below) since this track tends to stabilize early.

Person 5 — Backend API & Real-Time Layer

Owns: The API surface everyone else's data flows through, plus the standards-compliance layer.

Tech: FastAPI, WebSockets **Endpoints:** `/model` `/observations` `/fused` `/alerts` (REST) + WebSocket push for live alerts **Also owns:** Open Standards Layer — OGC WMS/WCS + OPeNDAP, CF Conventions compliance (this was a Problem Statement requirement missing from the original draft — don't skip it)

Day	Task
1	Repo/project setup. Stub all 4 REST endpoints against the agreed contract shapes. Scaffold the WebSocket connection.
2	Bring endpoints live as Person 4 and Person 6's real data lands. Start the Open Standards Layer wrapper.
3	Wire WebSocket to Person 2's real <code>/alerts</code> output for live push. Finish Open Standards Layer.
4	Likely one of the 3 people pulled onto frontend for Day 4 — owns wiring the live endpoints into the visualization.

Person 6 — In-Situ Data Pipeline + Adapter Pattern

Owens: Ingesting the real, ground-truth sensor data — the "Version B" side of the digital twin — plus the extensibility design.

Sources: Argo / Glider via Ifremer FTP; general in-situ via NOAA WOD **Also owns:** Adapter Pattern — a clean interface so new sensor types (moorings, HF-radar, ADCP) can be plugged in later without a rewrite (this directly answers the Problem Statement's "Extensible Design" requirement)

Day	Task
1	Design the adapter interface first (with Person 4 reviewing, since both tracks feed the same downstream layer). Start Ifremer FTP fetch scaffolding.
2	Get real in-situ data flowing through Person 5's <code>/observations</code> endpoint
3	Extend to multi-variable in-situ data (salinity, chlorophyll) where available; bug-fix the pipeline against edge cases (missing readings, irregular timestamps)
4	Bug fixes. Likely one of the 3 people pulled onto frontend for Day 4.

Day-by-day summary (all tracks)

Day 1 (Sept 5) — Contracts + scaffolding Start with a 30–45 min group sync: lock the exact JSON shape of all 4 endpoints, confirm region (Bay of Bengal) and the 5-variable set. Everyone then builds against that contract independently.

Day 2 (Sept 6) — Real data starts replacing mocks Model and in-situ pipelines go live; intelligence engines still running against mocks; API endpoints come online as real data lands.

Day 3 (Sept 7) — Full integration, no UI yet Everything wires together into one real loop: real model data → real in-situ data → Graph Fusion → Drift Memory → alerts, all testable via curl/Postman. **End-of-day target: the full backend loop runs with zero UI.**

Day 4 (Sept 8) — Frontend blitz + record + rehearse 3 people build the CesiumJS/Three.js visualization (scene, trust overlay, live alert feed, time-scrubber) and wire it straight to the now-real endpoints. The other 3 do final bug-fixes, threshold tuning, a stability pass, and prep the backup demo video + pitch script.

De-risking tip: whoever ends up on the Day 4 frontend team should spend an hour on Day 1 wiring a bare CesiumJS scene to mock data, so Day 4 is "swap in real endpoints and polish," not "build from zero."

Fallback order if time runs short (never cut the core loop)

1. Nowcast Engine — cut first, becomes a "Future Scope" slide bullet
2. Full PyTorch Geometric GNN — fall back to simpler graph-weighted correction
3. Expert/Public mode toggle — fall back to expert view only
4. Multi-variable support — fall back to SST only if currents/waves aren't stable in time
5. **Never cut:** real model data + real in-situ data + at least one visible fusion signal